Connect Littlebird (the two-way bridge)

Let your coding helper read your Littlebird briefs and chats, and write project summaries back to her, without leaking anything private.

Kind Walkthrough | Changed 2026-11-04 | For Anyone who runs Littlebird and a coding helper

You are here: drasticstatic lab › `my-template` › `walkthroughs` › `CONNECT-LITTLEBIRD.md` · public · `main` is the working branch

TL;DR

1. **Littlebird is an AI assistant that watches your screen, meetings and chats so it knows what you're working on.** Connected to your coding helper, the bridge runs both ways:
2. **Your helper reads Littlebird:** past conversations, routine reports (like a morning brief), screen context.
3. **Your helper writes to Littlebird:** short project summaries, so her next brief is accurate.
4. **Connect once per computer.** After that every project on that computer has it.

> ***Handing this to your AI helper?*** *Say: "Read CONNECT-LITTLEBIRD.md and help me connect Littlebird to you. Stop and ask me before anything that needs a login."*

Pick your helper

Claude Code (terminal)

1. Register the server for every project on this computer:

```
claude mcp add -s user --transport http littlebird https://mcp.littlebird.ai/mcp
```

2. **Start a new session.** A server added during a session isn't loaded until the next one starts (`claude --resume <name>` brings your conversation back).
3. Type `/mcp` , choose **littlebird**, choose **Authenticate**, and approve in the browser. There is no key to paste; it's a normal sign-in.
4. Check it: ask your helper to *"list my three most recent Littlebird conversations."*

If the terminal says *"The current working directory was deleted"* when you resume, the folder you're in was moved or re-created while the shell sat inside it: `cd` out and back in, then resume again.

Other helpers

Any helper that supports remote MCP servers with sign-in can use the same address, `https://mcp.littlebird.ai/mcp` . Littlebird's own docs (`docs.littlebird.ai`) are the authority on setup; trust them over anything a chat says about her login.

What your helper can do with it

Kind	Tools (names start `mcp__littlebird__`)	Ask before using?
Read conversations	`LB_INTERNAL_LIST_RECENT_CONVERSATIONS`, `…_SEARCH_CONVERSATIONS`, `…_READ_CONVERSATION`	no
Read routines (briefs)	`LB_INTERNAL_LIST_ROUTINES`, `…_GET_ROUTINE_CONFIG`, `…_GET_ROUTINE_REPORTS`	no

Read meetings	`LB_INTERNAL_LIST_MEETINGS`, `…_SEARCH_MEETINGS`, `…_GET_MEETING`	no
Read everything else	`search_user_context` (screen snapshots, messages, notes)	no, but keep it narrow
Read action cards	`get_action_cards` (drafts waiting for *your* approval; nothing here sends them)	no
Meeting transcripts	`LB_INTERNAL_GET_MEETING_TRANSCRIPT`	**yes**
Write to her	`ingest`, `LB_INTERNAL_CREATE_ROUTINE`, `…_UPDATE_ROUTINE`, `update_action_card`	**yes**

To stop being asked about the read-only ones, add them to `permissions.allow` in your own `~/.claude/settings.json`. Leave the "yes" rows asking: those write into her memory or open the most private material.

The rules that keep the bridge safe

1. **Read narrow.** Give a topic and a short date range. One wide search over a day of screen snapshots can be 70,000+ characters, most of it unrelated.
2. **Her context is private.** Littlebird sees your whole life: work, family, money, health. Your helper takes only the project facts it needs, and **nothing read from her goes into a public repo, a public page, a commit message or a changelog.**
3. **Ingest deliberately.** Whatever is ingested becomes part of what she draws on from then on. Send short, project-level summaries you asked for; never keys, passwords, personal emails, or anything taken from her own private context. Keep a copy of each ingest in the repo it came from, so you can see what she was told.
4. **Check her against reality.** Her briefs are observations. Before acting on one, open the live page or the repo and confirm. If she reports something as "live" that you only saw committed, say so in the next ingest.
5. **Her notes have a home.** Each repo's `AGENT-SYNC/created-by-Littlebird/` holds her dated notes and a README written for that repo. If her GitHub app pushes there, it pushes only to that folder, pulls first, and never skips the safety checks.

What it's like when it works

She notices things (a new key on a repo at an odd hour, a deploy that failed overnight) and puts them in your morning brief. Your coding helper checks them, fixes what's real, and ingests a short "here's what's actually true now". The next brief starts from the truth.

Suggest a change

Noticed something wrong, or know a better way? Open an issue or a pull request against `main`; the lab steward reads them, and a person with the role merges with a merge commit so your name stays on it. Not on GitHub? Tell any steward you know, or your helper, in your own words.

One of the walkthroughs in `my-template`. Icons: Lucide (ISC).